Distribuez vos données avec MongoDB

L'architecture du sharding sous MongoDB

Voyons la question de la distribution pour le passage à l'échelle sous MongoDB. Cette distribution des blocs de données (*chunk*) appelée *sharding* est définie par un tri des données, contrairement à *HDFS* (placement physique) ou au *Consistent Hashing* (hachage sur un anneau). Ainsi, vous devez définir l'architecture physique de la distribution des serveurs appelée cluster.

La robustesse du *cluster* est gérée par 3 types de nœuds (serveurs) :

- Des routeurs : **mongos**
- Des serveurs de configuration : **Config Server**
- Des serveurs de données : **Shard**

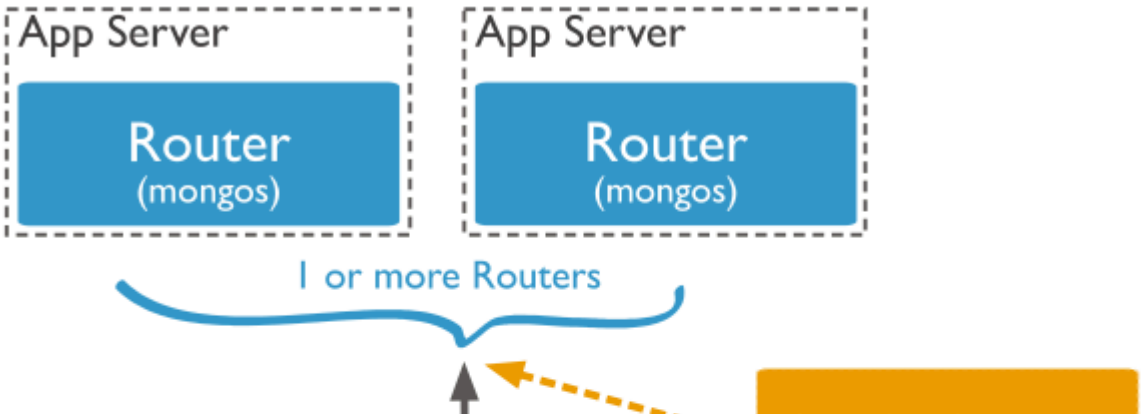
La composition minimum d'un cluster est de 2 *mongos*, 3 *Config Servers* et 2 *shards*.

Un **mongos** s'occupe du routage des requêtes. Il stocke les informations sur l'arbre permettant de répartir les données (voir le chapitre sur le sharding). Ainsi, bien qu'il soit un point central de l'architecture, le temps de traitement est rapide. Il faut minimum deux routeurs pour permettre de gérer la tolérance aux pannes de celui-ci.

Les **ConfigServers** s'occupent de la connaissance du réseau, aussi bien au niveau des routeurs que les *shards*. Ils vont ainsi stocker les informations sur l'arbre de routage utilisé par les routeurs ; permettant de synchroniser celui-ci et de l'intégrer à un nouveau routeur. Ils gèrent également les informations de répartition de charge sur les différents *shards*, et la structure des *ReplicaSets* (vu dans le chapitre précédent) permettant de rééquilibrer la charge des serveurs si la répartition des données est inégale. Les *ConfigServers* sont au nombre de 3 et organisés en *ReplicaSet* pour garantir l'intégrité du réseau. L'idéal est de les placer à des endroits clés du réseau pour des questions de pérennité du système (rack différents, proches des routeurs).

Les **shards** contiennent l'ensemble des données, les chunks. Ils peuvent contenir plusieurs chunks, mais pas forcément contigus (tri des données de l'arbre).

L'architecture de distribution d'un cluster est schématisée dans la figure ci-dessous :



Architecture du sharding sous MongoDB

Nous allons maintenant nous attaquer à la création de ce cluster. Pour son instanciation, il faudra suivre les étapes suivantes :

1. Lancer les **ConfigServer** en ReplicaSet (ici *configReplSet*)
2. Lancer chaque **Shard** en ReplicaSet (ici *sh1* et *sh2*)
3. Lancer un **mongos**
    1. Connecter les ConfigServer
    2. Connecter les shard
    3. Lancer le sharding sur une collection

Création du serveur de configuration

Pour créer le cluster, il vous faut tout d'abord un *ConfigServer*. Bien sûr, il est nécessaire de (**1**) créer un répertoire dédié pour chaque serveur *mongod* (pas pour les *mongos*), (**2**) définir un port d'écoute à chacun. Les instructions suivantes sont à exécuter dans des consoles différentes :

```
mongod --configsvr --replSet configReplSet --port 27019
--dbpath /data/config1 mongod --configsvr --replSet
configReplSet --port 27020 --dbpath /data/config2
```

Nous avons donc lancé deux serveurs de configuration (**--configsvr**) en ReplicaSet (**configReplSet**). Dans une troisième console, vous pouvez vous connecter à un de ces serveurs et initier le ReplicaSet :

```
mongo --port 27019
```

Unset

```
rs.initiate();
```

```
rs.add("local:27020");
```

Création des shards

Ensuite, il est nécessaire de lancer les shard en ReplicaSet. Nous allons en créer deux pour tester la distribution. Pour chaque shard, le paramètre --shardsvr est nécessaire pour permettre son intégration.

```
mongod --shardsvr --replSet sh1 --port 27031 --dbpath
/data/sh1 mongod --shardsvr --replSet sh2 --port 27032
--dbpath /data/sh2 mongo --port 27031 --eval "rs.initiate()"
mongo --port 27032 --eval "rs.initiate()"
```

L'option **--eval** permet de faire passer la commande **rs.initiate()** directement au serveur puis de récupérer la main sur la console. Ici, cette commande permet d'initialiser le *ReplicaSet* de chaque *shard*.

Lancement du mongos et connexion des shards

Maintenant que nous avons des shards et des ConfigServers, nous pouvons nous attaquer au mongos (routeur).

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

Il est possible de rajouter chacun des serveurs du ReplicaSet, en les séparant par des virgules :

```
configReplSet/localhost:27019,localhost:27020,localhost:27021
```

Les *shards* peuvent alors être ajoutés les uns après les autres au niveau du mongos en mode console :

```
mongo --port 27017
```

Unset

```
sh.addShard( "sh1/localhost:27031");
```

```
sh.addShard( "sh2/localhost:27032");
```

Distribution de la base de données

L'architecture de distribution est mise en place. Il suffit maintenant de définir la collection que l'on veut distribuer. Pour cela, nous allons créer une base « testDB » et une collection « test ».

```
Unset

use testDB;

sh.enableSharding("testDB");

db.createCollection("test");

db.test.createIndex({"_id":1});

sh.shardCollection("testDB.test",{"_id":1});
```

La collection test est maintenant distribuée automatiquement sur nos deux shards, les documents sont placés dans les *chunks* en fonction de leur valeur d'identifiant "_id", un tri de ces valeurs est effectué pour déterminer le placement (cf sharding arborescent). Nous pouvons maintenant importer les données des restaurants dans cette base de test :

```
mongoimport --db testDB --collection test --port 27017
restaurants_NY.json mongo --port 27017 --eval "sh.status()"
```

```
Unset

--- Sharding Status ---

sharding version: {

"_id" : 1,

"minCompatibleVersion" : 5,
```

```
"currentVersion" : 6,

"clusterId" : ObjectId("5967a3f7695a5111518224c6")

}

shards:

{  "_id" : "sh1",  "host" : "rs0/local:27031",  "state" : 1 }

{  "_id" : "sh2",  "host" : "rs1/local:27032",  "state" : 1 }

active mongoses:

"3.4.3" : 1

autosplit:

Currently enabled: yes

balancer:

Currently enabled:  yes

Currently running:  no
```

```
      Balancer lock taken at Thu Jul 13 2017 18:46:47 GMT+0200 (CEST) by
ConfigServer:Balancer

Failed balancer rounds in last 5 attempts:  0

Migration Results for the last 24 hours:

      2 : Success

databases:

{  "_id" : "testDB",  "primary" : "rs0",  "partitioned" : true }

      testDB.test

            shard key: { "_id" : 1 }

            unique: false

            balancing: true

            chunks:

                  sh1     2
```

```
                    sh2     2

            { "_id" : { "$minKey" : 1 } } -->> { "_id" :
ObjectId("5967a455b6c0223b91eefbd1") } on : rs1 Timestamp(2, 0)

            { "_id" : ObjectId("5967a455b6c0223b91eefbd1") } -->> { "_id" :
ObjectId("5967a455b6c0223b91eeffba") } on : rs1 Timestamp(3, 0)

            { "_id" : ObjectId("5967a455b6c0223b91eeffba") } -->> { "_id" :
ObjectId("5967a455b6c0223b91ef01af") } on : rs0 Timestamp(3, 1)

            { "_id" : ObjectId("5967a455b6c0223b91ef01af") } -->> { "_id" :
{ "$maxKey" : 1 } } on : rs0 Timestamp(1, 4)
```

Nous pouvons constater dans la partie « **databases** » que la collection *testDB.test* est composée de 4 *chunks*, répartis uniformément sur les deux *shards*.

> *enableSharding* ne marche **que** sur une collection préalablement **vide**. Toute collection déjà existante ne peut être distribuée. Il faudra ainsi importer les données dans la collection « *shardée* ».

Stratégie de distribution

Nous avons vu comment créer une architecture de distribution et comment distribuer une collection. Maintenant, il serait intéressant de savoir comment choisir une stratégie de distribution appropriée à nos données, afin de fournir les meilleures performances pour nos requêtes.

Requêtes par intervalles

Jusqu'ici, nous nous sommes contentés de choisir l'identifiant « **_id** » pour distribuer les données. Par défaut, l'indexation est basée sur une index non-dense triant les données. De fait, les requêtes par intervalles de valeurs sont optimisées. Il est donc

possible de choisir une autre clé pour les documents de la collection comme stratégie de distribution.

Pour profiter des requêtes par intervalles, nous pourrions créer une « clé de sharding » sur le code postal, la clé : *zipcode*. Ainsi, toute requête cherchant à retrouver les restaurants associés à ce code postal ou une plage de codes postaux seront efficaces.

> Tout changement de stratégie de sharding impose la création d'une nouvelle collection. On ne peut changer la distribution d'une collection déjà « shardée ».

```
Unset

sh.shardCollection("testDB.test2", {"address.zipcode" : 1});
```

Requêtes par zones

Le *zipcode*, c'est bien, mais comment mieux maîtriser la répartition des restaurants sur les différents *chunks* ? En effet, la technique précédente ne permet pas de contrôler la répartition physique des *chunks* et les intervalles de valeurs de ceux-ci. Le *sharding* par zone permet d'associer à chaque *shard* une zone. Tous les restaurants de cette zone seront alors alloués à un serveur dédié. L'avantage est double :

1. **Optimiser** toutes **requêtes** liées à une **zone** (filtrage ou agrégation par quartier).
2. Définir un **serveurphysique** dans un *datacenter***proche** du client (zone géographique), correspondant à des requêtes de proximité.

Pour cela, il faut tout d'abord définir ces zones. Une zone est composée d'une plage de valeur et d'un « tag ». Nous pouvons choisir sur un intervalle de codes postaux correspondant aux villes :

```
Unset

sh.addTagRange("testDB.test2",

  {"address.zipcode": "10001"}, {"address.zipcode":  "11240"},
```

```
    "NYC")

sh.addTagRange("testDB.test2",

    {"address.zipcode": "94102"}, {"address.zipcode":  "94135"},

    "SFO")
```

L’avantage est de pouvoir associer des *shards* physiquement proches des utilisateurs dans chaque ville pour réduire la latence du réseau. Cela peut avoir également un intérêt si l’on souhaite stocker des données dans un cluster sécurisé.

Attention toutefois à la surcharge d’un *shard* suite à une zone mal dimensionnée. Nous pouvons découper une zone sur deux tags.

Unset

```
sh.addTagRange("testDB.test2",

    {"address.zipcode": "10001"}, {"address.zipcode": "10281"},

    "NYC-Manhattan")
```

```
sh.addTagRange("testDB.test2",
  {"address.zipcode": "11201"}, {"address.zipcode": "11240"},
  "NYC2-Brooklyn")
```

Pour information, la plage de valeurs inclut le min (10001), mais pas le max (10281).

Maintenant que les tags sont associés à des plages de valeurs, il suffit d'associer à chaque tag un shard. Un tag peut être associé à plusieurs shards.

Unset

```
sh.addShardTag("sh1", "NYC")
sh.addShardTag("sh2", "NYC")
```

```
sh.addShardTag("sh2", "SFO")
```

Automatiquement, les restaurants seront placés dans les *chunks* des *shards* désignés en fonction de leurs codes postaux. Il faudra toutefois rester vigilant à la répartition de charge et le dimensionnement des *shards*. Pour cela, vous pouvez consulter la répartition des *chunks* d'un tag dans le réseau :

```
Unset
use config;

db.shards.find({ "tags" : "NYC" });
```

Requêtes par catégories

Et si nous faisions des requêtes ciblées sur un certain type de valeurs comme le quartier (*borough*) ? Il n'y a dans ce cas pas de plages de valeurs possibles, uniquement des restaurants correspondant à un quartier donné. Un simple partitionnement par hachage suffirait amplement à la segmentation des données.

MongoDB offre la possibilité de faire un hachage (via la fonction *md5*) sur les valeurs de la clé de sharding. L'avantage est soit de pouvoir **contrôler le placement** de tous les documents partageant la même valeur, soit de faire une **répartition plus uniforme** des documents sur l'ensemble des shards.

```
Unset
sh.shardCollection( "testDB.test3", { "borough" : "hashed" } )
```

Attention, le routeur n’est alors efficace que si l’on utilise des requêtes d’égalité sur le quartier. Les requêtes par intervalles ne permettent pas d’effectuer un routage sur les *shards*. De fait, la requête est transmise à tous les *shards* par *broadcast*.

Index et performances

Bien entendu, nous ne pouvons choisir qu’une seule stratégie de distribution pour optimiser nos requêtes. Mais MongoDB offre la possibilité d’interroger n’importe quelle clé. Comment éviter alors le *broadcast* lorsque la clé de *sharding* n’est pas utilisée ?

La collection « *testBD.test* » est distribuée sur l’identifiant des restaurants « *_id* ». Je souhaite maintenant exécuter la requête suivante :

```
Unset

db.test.find({"address.street" : "3 Avenue"}).explain()

{
  "queryPlanner" : {
    "mongosPlannerVersion" : 1,
    "winningPlan" : {
      "stage" : "SHARD_MERGE",
      "shards" : [
        {
```

```
        "shardName" : "sh1",
        "connectionString" : "sh1/local:27030",
        "serverInfo" : {
          "host" : "local",
          "port" : 27031,
          "version" : "3.4.3",
          "gitVersion" : "f07437fb5a6cca07c10bafa78365456eb1d6d5e1"
        },
        "plannerVersion" : 1,
        "namespace" : "testDB.test",
        "indexFilterSet" : false,
        "parsedQuery" : {
          "address.street" : {
```

```
            "$eq" : "3 Avenue"

          }

        },

        "winningPlan" : {

          "stage" : "SHARDING_FILTER",

          "inputStage" : {

            "stage" : "COLLSCAN",

            "filter" : {

              "address.street" : {"$eq" : "3 Avenue"}

            },

            "direction" : "forward"

          }

        },
```

```
        "rejectedPlans" : [ ]
    },
    {
        "shardName" : "sh2",
        "connectionString" : "sh2/local:27031",
        "serverInfo" : {
            "host" : "local",
            "port" : 27032,
            "version" : "3.4.3",
            "gitVersion" : "f07437fb5a6cca07c10bafa78365456eb1d6d5e1"
        },
        "plannerVersion" : 1,
        "namespace" : "testDB.test",
```

```
        "indexFilterSet" : false,
        "parsedQuery" : {
          "address.street" : {
            "$eq" : "3 Avenue"
          }
        },
        "winningPlan" : {
          "stage" : "SHARDING_FILTER",
          "inputStage" : {
            "stage" : "COLLSCAN",
            "filter" : {
              "address.street" : {
                "$eq" : "3 Avenue"
```

```
            }
            },
            "direction" : "forward"
        }
        },
        "rejectedPlans" : [ ]
    }
    ]
  }
  },
  "ok" : 1
}
```

La fonction "**.explain()**" appliquée à une requête permet de consulter le plan d'exécution généré. Ici, le plan « *WinningPlan* » possède une étape (stage) « *SHARD_MERGE* » qui effectue la fusion des résultats provenant de deux « *COLLSCAN* » (« *Collection Scan* »), en l'occurrence un scan complet de la collection pour chaque *shard*. Cela correspond à une lecture complète du contenu des chunks, ce qui n'est pas très efficace.

Nous allons créer un index sur la rue :

```
Unset

db.test.createIndex({"address.street" : 1})
```

Après avoir testé à nouveau la requête, l'index est utilisé car l'étage passe en « *IXSCAN* ». Ce qui va permettre d'accéder directement aux restaurants de la 3° avenue sans avoir à parcourir toute la base de données. Bien sûr, comme ce n'est pas la clé de *sharding*, tous les *shards* effectuent cette recherche. Même si un seul restaurant correspond à un critère de recherche, tous les *shards* parcourent l'index. Il n'y a pas d'index global, uniquement des indexes locaux à chaque *shard*. Toutefois, les performances seront améliorées.

Si l'on souhaite profiter des avantages d'un index dans le cadre d'une requête « *aggregate* », il faut que le premier critère de recherche soit un filtre sur la clé utilisée dans l'index. Dans notre cas, seul un opérateur de type **$match** contenant « *address.street* » permettrait l'utilisation de l'index (idem pour la clé de *sharding*).

Pour consulter le plan d'exécution d'un « *aggregate* », le paramètre *{explain:true}* doit être ajouté après la séquence d'opérations :

```
Unset

db.test.aggregate([

  {$match:{"address.street" : "3 avenue"}},
```

```
  {$group:{_id:"$borough", total : { $sum : 1}}}

],

 {explain:true}

};
```